

GOOGLE FILE SYSTEM

Ruitao Xie

<https://rtxie.github.io>

GOOGLE FILE SYSTEM

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. 2003. The Google file system. In SOSP '03. ACM, New York, NY, USA, 29–43.

Google Citation: 10526

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

Why Build GFS?

- ❑ Need a scalable, distributed file system that targets Google's workloads
 - Need to support much bigger (GB+) files
 - On 100/1000s of commodity servers that fail regularly

GFS: WORKLOAD ASSUMPTIONS

- ❑ “Modest” number of large files: $\sim 10^6$ files, each $\sim 100\text{MB}$
- ❑ Two kinds of reads: Large streaming (100s KB or 1MB) and small random (few KB)
- ❑ Writes: Most writes are file appends, Overwrites are practically non-existent
- ❑ High bandwidth more important than low latency

GFS: DESIGN

- ❑ Single Master for metadata
- ❑ Chunkservers for storing data
- ❑ No POSIX API !
- ❑ No Caches!

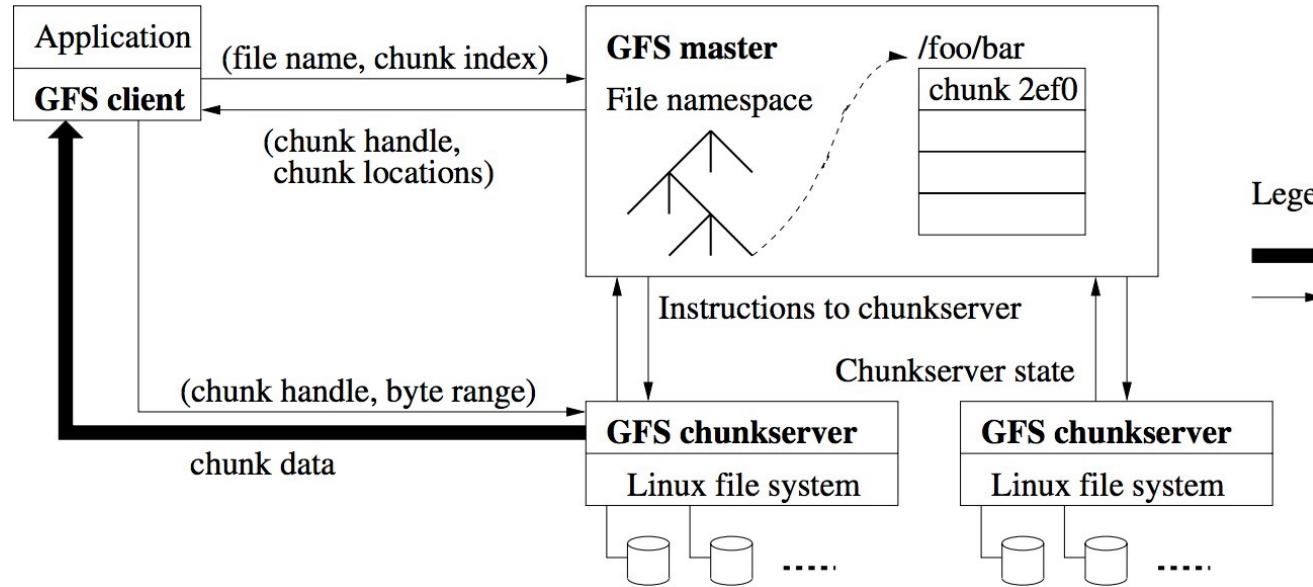
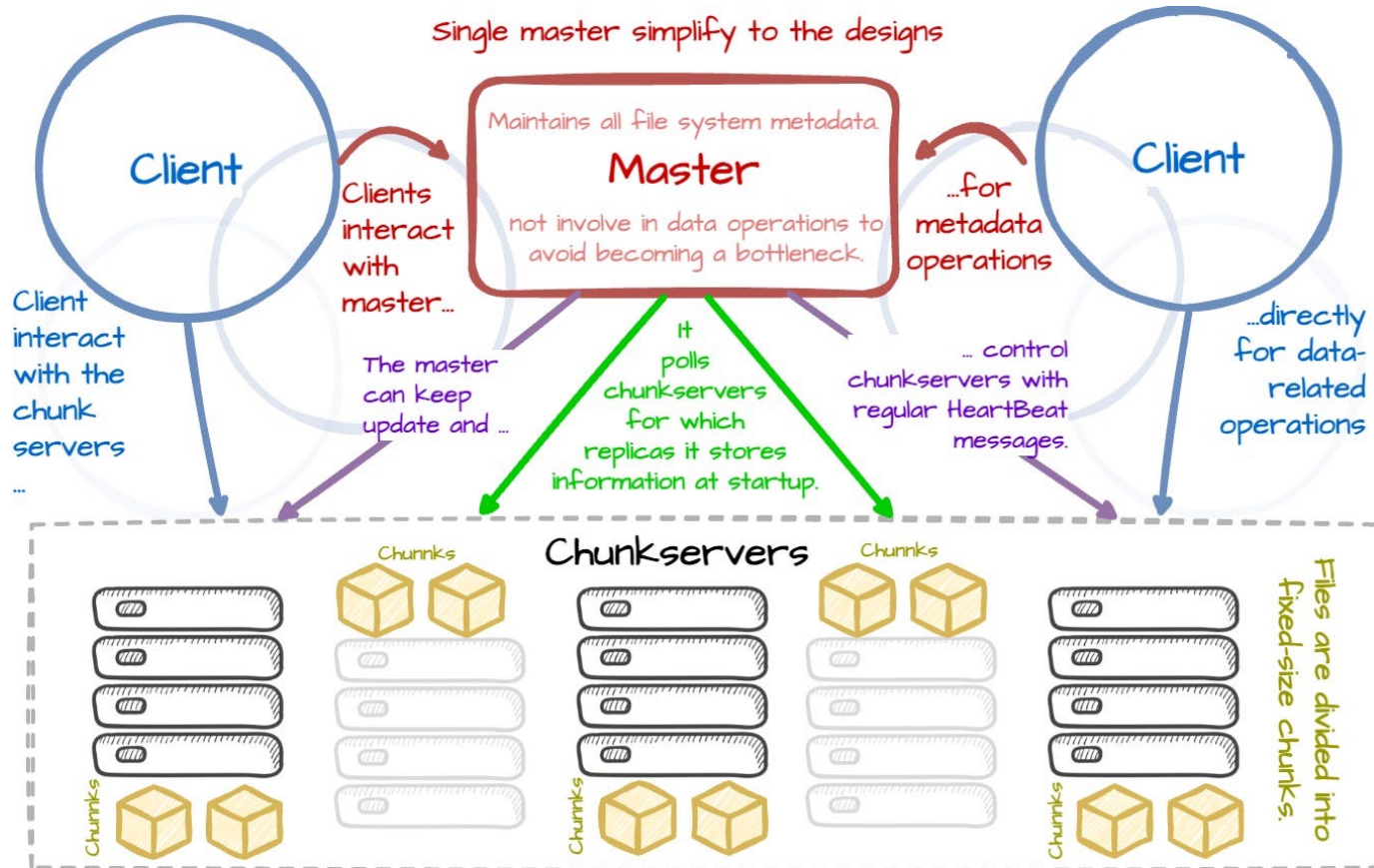
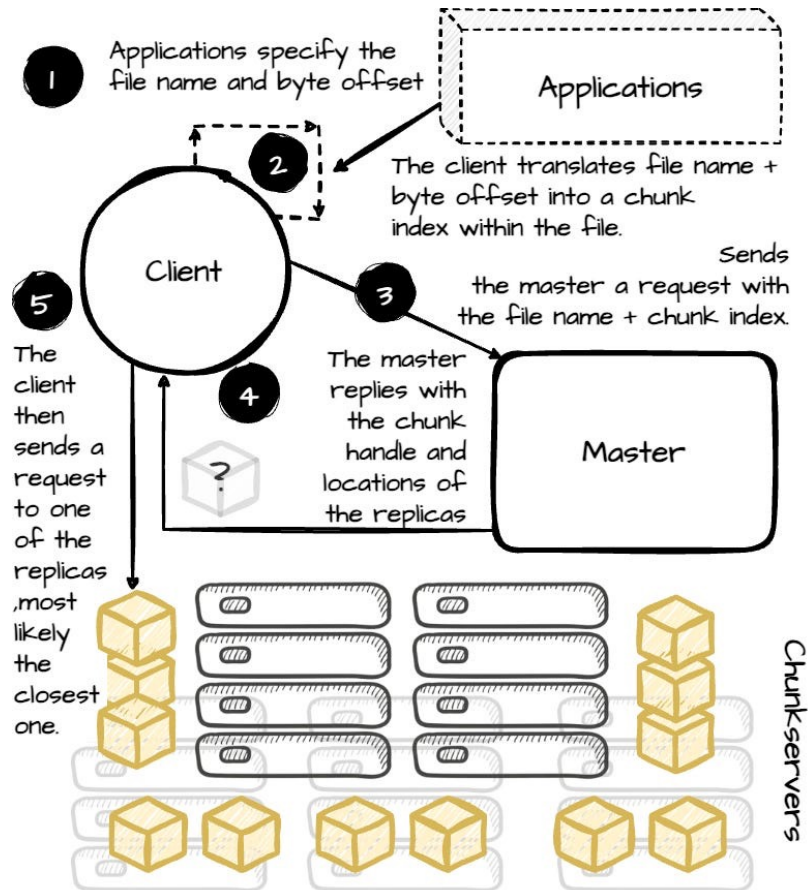


Figure 1: GFS Architecture

GFS'S HIGH-LEVEL ARCHITECTURE



THE INTERACTIONS FOR A SIMPLE READ



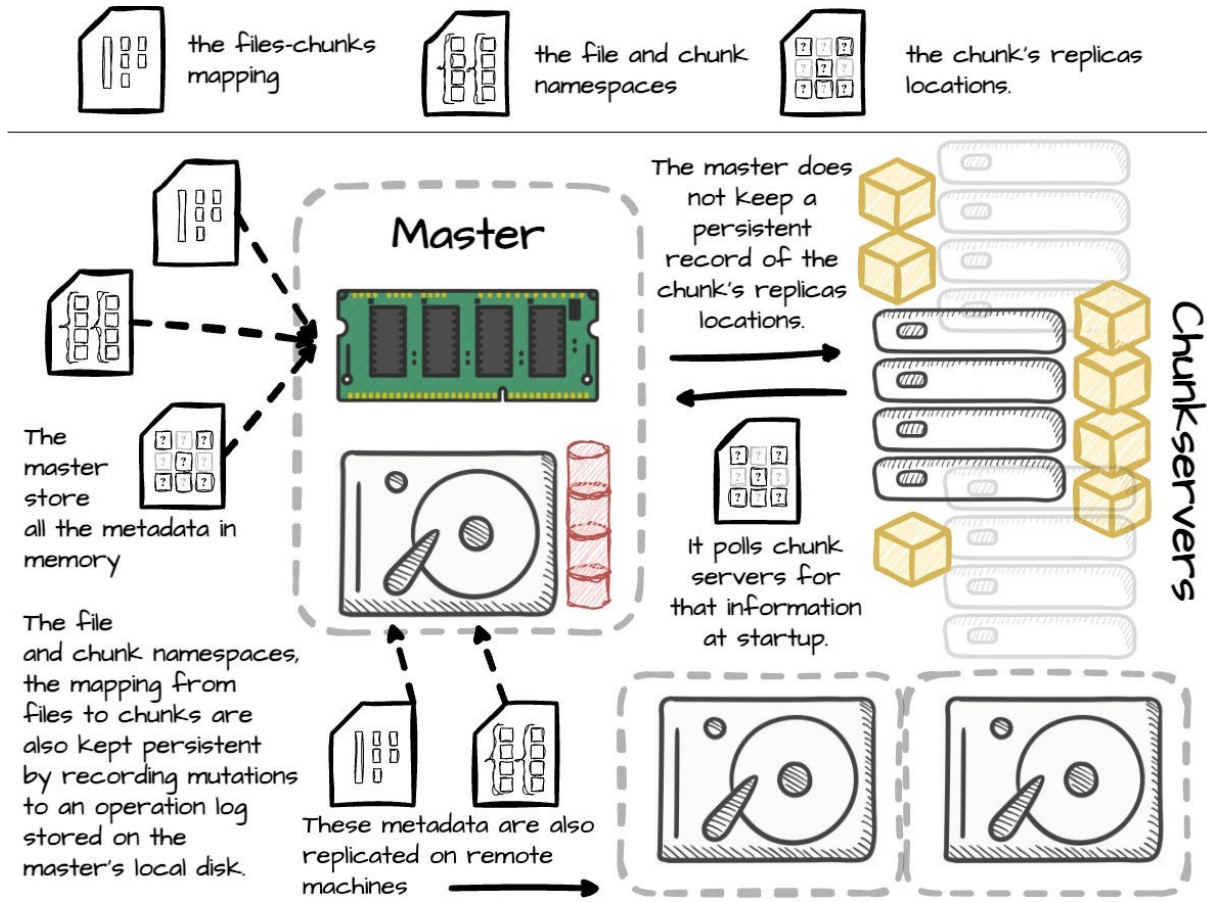
CHUNK SIZE (64 MB) TRADE-OFFS

❑ Advantage of large chunk size:

- Client to Master: request for chunk location before each read/write
- Client to Chunk Server: reduce network overhead via a persistent TCP connection
- Metadata: keep it in memory

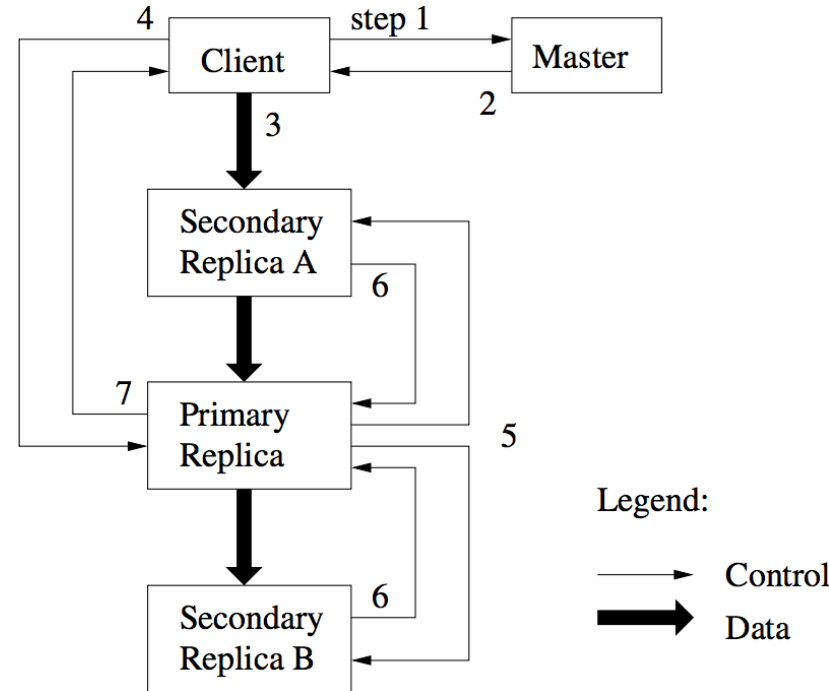
❑ Disadvantage of large chunk size: hot spot issue

METADATA



GFS: REPLICATION

- ❑ 3-way replication to handle faults
- ❑ Primary replica for each chunk
- ❑ Chain replication (consistency)
- ❑ Decouple data, control flow
- ❑ Dataflow: Pipelining, network-aware

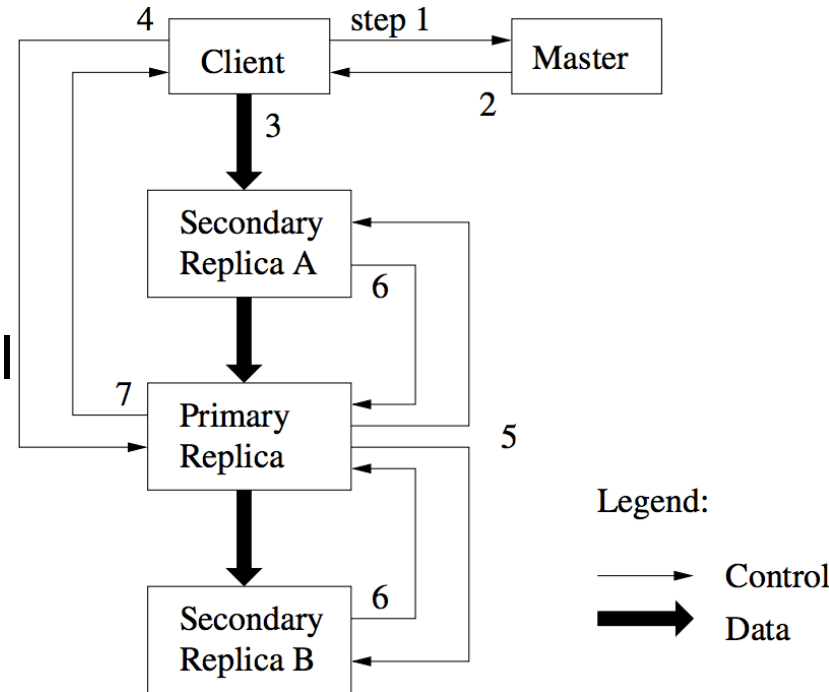


GFS: REPLICATION

Given T is the network throughput and L is latency to transfer bytes between two machines,
the ideal elapsed time for transferring B bytes to R replicas?

Chunk Write Implementation

1. Ask for chunk lease & request for locations
2. Primary & secondary locations
3. Push data to replicas, cache data in LRU buffer
4. Request write, primary assigns serial numbers to mutations
5. Request secondary write in the same order
6. Reply when write is done
7. Ack. If any errors, repeat 3-7



RECORD APPENDS

Scenario: many clients on different machines append to the same file concurrently

Write: Client specifies the offset

Record Append: GFS chooses offset

Consistency

- At-least once
- Atomic

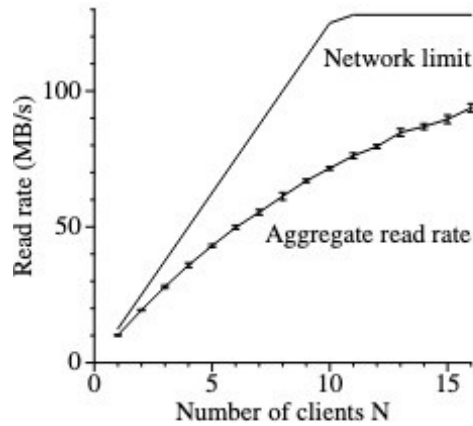
REPLICA PLACEMENT CONSIDERATIONS

- ☐ Disk space utilization
- ☐ Spread replicas of a chunk across racks
- ☐ Limit the number of “recent” creations on each chunkserver

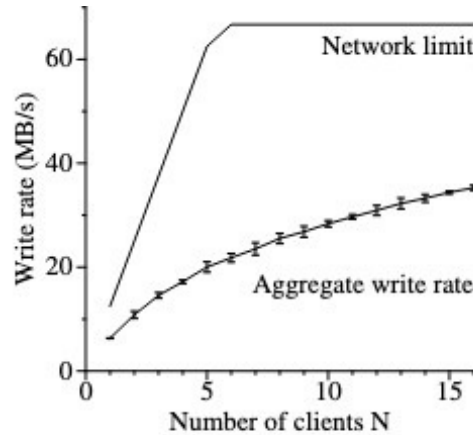
FAULT TOLERANCE

- ❑ Chunk replication with 3 replicas
- ❑ Master
 - ❑ Replication of log, checkpoint
 - ❑ Shadow master
- ❑ Data integrity using checksum blocks

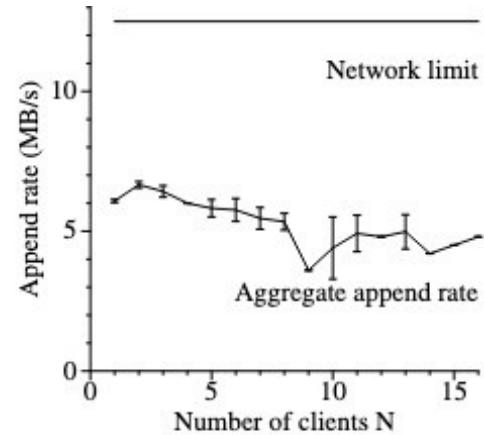
What happens with a faster network (125MB/s) but same disks (100 MB/s)?



(a) Reads



(b) Writes



(c) Record appends

WHAT HAPPENED NEXT



Cluster-Level Storage @ Google

How we use *Colossus* to improve storage efficiency

Denis Serenyi

Senior Staff Software Engineer

dserenyi@google.com

Keynote at PDSW-DISCS 2017: 2nd Joint International Workshop On Parallel Data Storage
& Data Intensive Scalable Computing Systems

GFS EVOLUTION

Motivation:

- ❑ GFS Master

 - One machine not large enough for large FS

 - Single bottleneck for metadata operations (data path offloaded)

 - Fault tolerant, but not HA

- ❑ Lack of predictable performance

 - No guarantees of latency

 - (GFS problems: one slow chunkserver -> slow writes)

GFS EVOLUTION

GFS master replaced by Colossus

Metadata stored in BigTable

Recursive structure ? If Metadata is $\sim 1/10000$ the size of data

100 PB data $\rightarrow$ 10 TB metadata

10TB metadata $\rightarrow$ 1GB metametadata

1GB metametadata $\rightarrow$ 100KB meta...

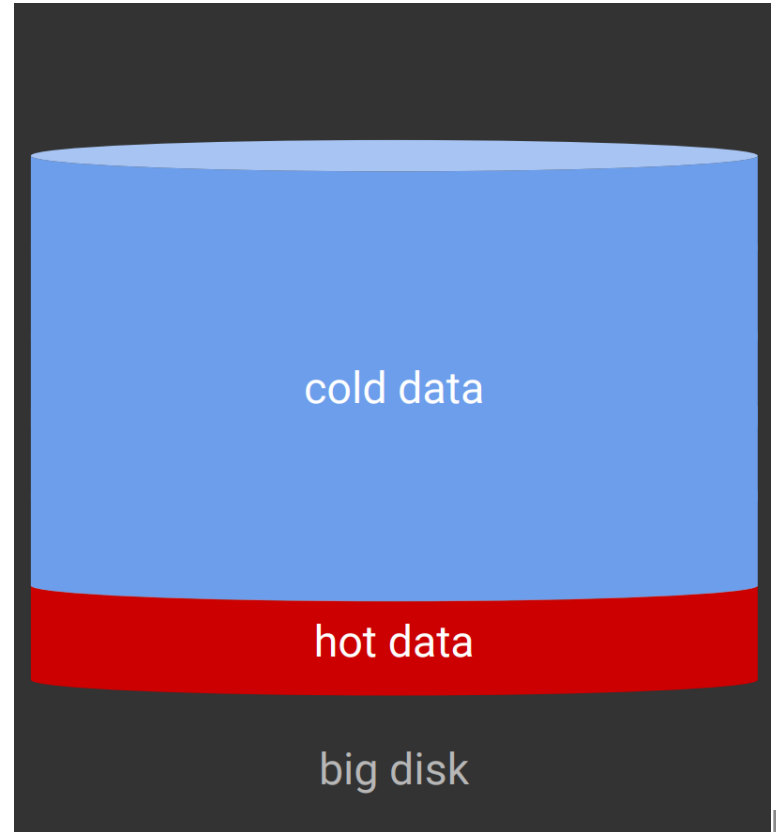
GFS EVOLUTION

Need for Efficient Storage

Rebalance old, cold data

Distributes newly written data evenly across disk

Manage both SSD and hard disks



RESEARCH PROBLEM

<https://rtxie.github.io/en/publication/18-tcc/18-TCC.pdf>

IEEE TRANSACTIONS ON CLOUD COMPUTING, VOL. 6, NO. 1, JANUARY-MARCH 2018

Data Transfer Scheduling for Maximizing Throughput of Big-Data Computing in Cloud Systems

Ruitao Xie and Xiaohua Jia , *Fellow, IEEE*

RESEARCH PROBLEM

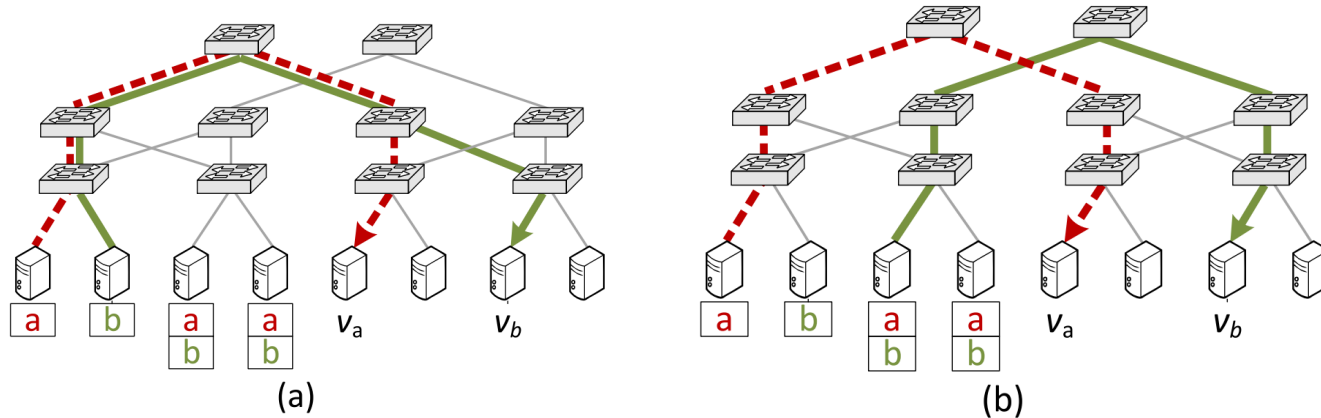
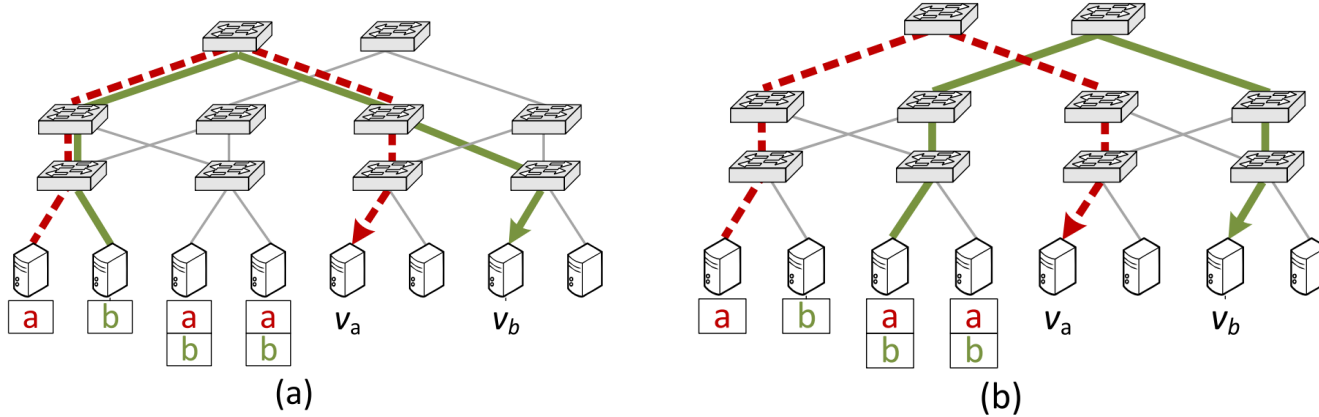


Fig. 1. An example to motivate the optimal data retrieval method. Node v_a is retrieving data a (red dash lines) and v_b is retrieving data b (green solid lines). In (a) both data transfers share common links, which has more traffic and may lead to longer transmission time, while in (b) they pass through disjoint sets of links, resulting in shorter data retrieval time.

RESEARCH PROBLEM



Data retrieval problem: to select nodes and paths such that the data retrieval time of an application is minimized.

THANKS